

CodeFab

Designed by **A**rrange, **A**ct, **A**ssert in DongTan

Roles & Responsibilities



윤지영 님

- Tokenizer
- Shell mode - File



이상훈 님

- AssemblerUnit
 - var, for, import
- SystemTest



이승훈 님

- AssemblerUnit
 - expression, block, class
- Shell mode - Debug



최상범 님

- AssemblerUnit
 - print, if, func
- Shell mode - Debug



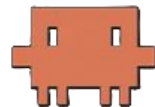
김대진 님

- CheckerUnit
- Documentation



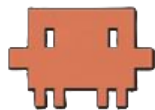
장종인 님

- ExecutorUnit



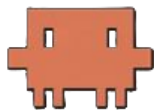
Claude

Roles & Responsibilities



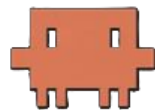
윤지영 님 Claude

- Tokenizer
- Shell mode - File



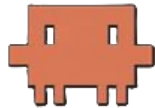
이상훈 님 Claude

- AssemblerUnit
 - var, for, import
- SystemTest



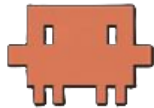
이승훈 님 Claude

- AssemblerUnit
 - expression, block, class
- Shell mode - debug



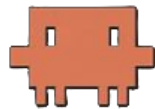
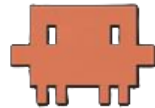
최상범 님 Claude

- AssemblerUnit
 - print, if, func
- Shell mode - Debug



김대진 님 Claude

- CheckerUnit
- Documentation



장종인 님

- ExecutorUnit

Claude Claude

Architecture Pattern: Pipeline

Source -> **Tokenizer** -> **AssemblerUnit** -> **Parser Registry** -> **AST** -> **CheckerUnit** -> **ExecutorUnit**

Tokenizer

입력 문자열을 TokenList로 변환합니다. import 문을 만나면 대상 파일을 재귀적으로 splice합니다.

AssemblerUnit

TokenList를 순회하며 ProgramNode AST를 조립합니다.

Registry

현재 TokenType에 맞는 IStatementParser 구현체를 선택합니다.

CheckerUnit

AST 의미 검사와 정적 정보 기록을 수행합니다.

ExecutorUnit

AST를 실행하며 runtime scope와 value를 관리합니다.

AssemblerUnit : Parser Architecture

AssemblerUnit

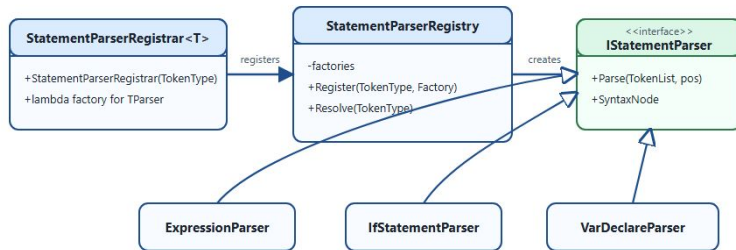
- > tokenList[pos].type 확인
- > StatementParserRegistry::Resolve(tokenType)
- > IStatementParser::Parse(tokenList, pos)
- > SyntaxNode 반환

Registry Pattern

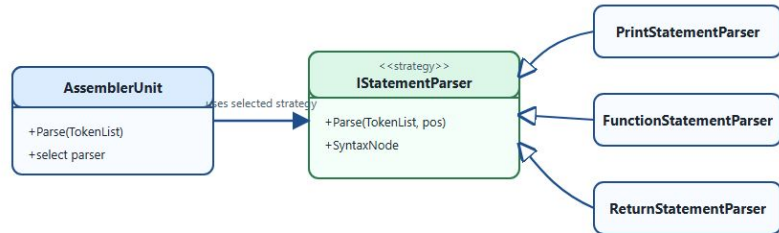
Factory Pattern

Strategy Pattern

Registry + Factory Pattern



Strategy Pattern



Abstract Syntax Tree

```
SyntaxNode
- NodeType type
- Token token
- vector<unique_ptr<SyntaxNode>> children
- Accept(NodeVisitor&)
```

노드	children 의미
BinaryExprNode	[left, right]
AssignExprNode	[target, value]
IfStmtNode	[condition, thenBranch, optionalElseBranch]
ForStmtNode	[init, condition, increment, body]
FuncDeclStmtNode	[param..., body]
ClassDeclStmtNode	[method(FuncDeclStmtNode)...] (+ parentNameToken)
ImportStmtNode	[path(StringLiteralNode), alias(IdentifierNode)]
MemberAccessExprNode	[target] (+ token = 멤버 이름)

Checker/ExecutorUnit

```
node->Accept(visitor)
-> visitor.Visit(concreteNode)
```

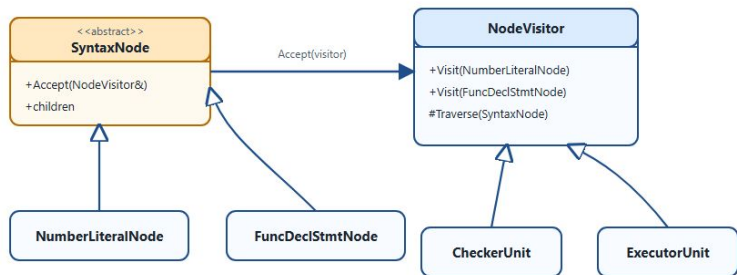
CheckerUnit

- 중복 선언 검사
- return 위치 검사
- this/super 위치 검사
- 정적 바인딩 거리 기록
- 상수 폴딩 결과 기록

ExecutorUnit

- statement 실행
- expression 평가
- runtime scope 관리
- 함수/배열/클래스 실행
- 출력과 런타임 에러 처리

Visitor Pattern



BP#1. TDD. Automated by Claude Code

Claude Code System Test Skill

콘솔 인터프리터를 자동 검증하는 회귀 테스트 스킴

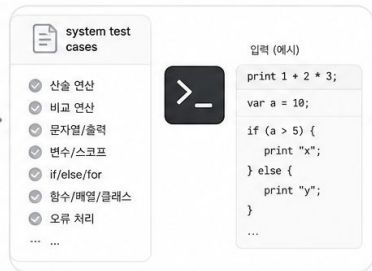
1. Build (1회)

Release 빌드 1회



2. Run (N회)

정의된 케이스를 순차 실행



3. Compare

기대 출력과 비교



Test Cases



콘솔 사용 시나리오를
카테고리별로 체계화

Case Example

InputLines

```
@(  
  "var a = 10;"  
  "a = a + 5;"  
  "print a;"  
)
```

Expect

15

여러 줄 입력 지원
(실제 콘솔 입력 순서 그대로)

Error Case

InputLines

```
@("print unknownVar;")  
  
ExpectError = $true
```

기대 출력 대신 에러 발생 여부만
검증 (메시지 무관)



Version Controlled

모든 테스트 케이스는 run.ps1에 정의되어 저장소에서 형상 관리됩니다.

→ 케이스 추가 / 수정은 코드

CodeFab 모듈별 테스트 커버리지

UnitTC(324) + SystemTC(77) 중합 · 라인 커버리지 · 전체 평균 89.5%



90%+

70~89%

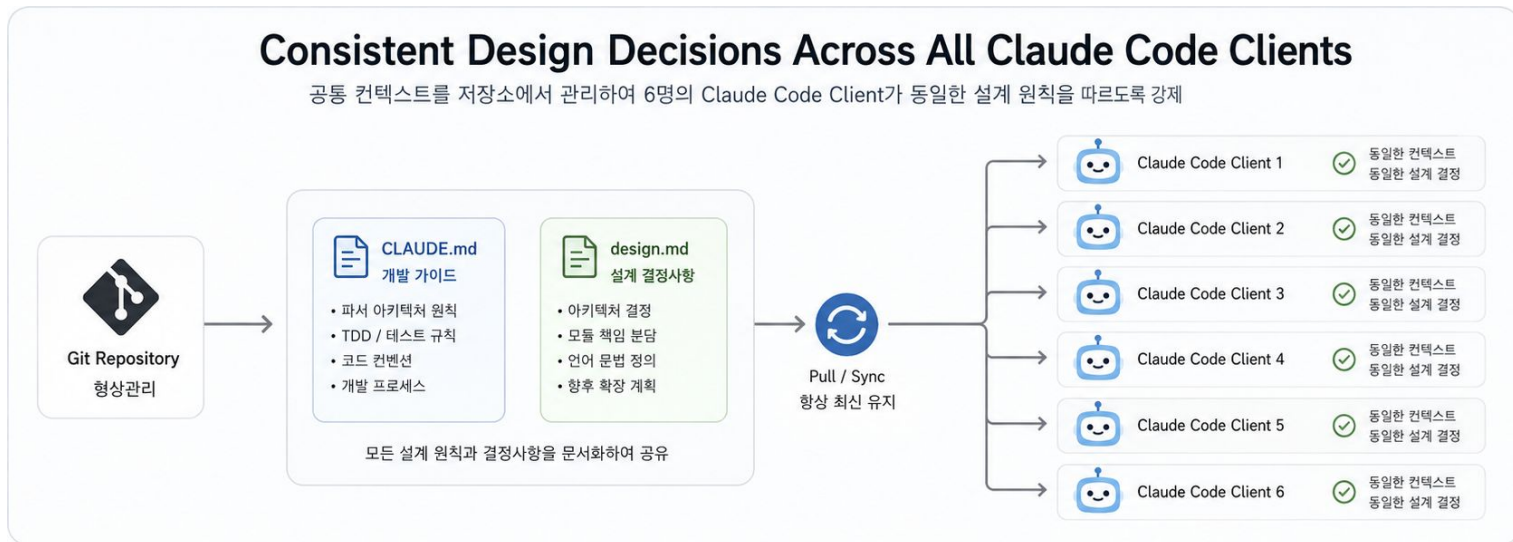
40~69%

0~39%

BP#2. A.I. Consistent Design Decisions

Consistent Design Decisions Across All Claude Code Clients

공통 컨텍스트를 저장소에서 관리하여 6명의 Claude Code Client가 동일한 설계 원칙을 따르도록 강제



일관성 보장

누구의 Claude Code를 사용하든
같은 설계 원칙을 따릅니다



지식 공유

문서화된 결정사항이
팀 자산으로 축적됩니다



의사결정 가속

명확한 기준으로 빠르고
일관된 의사결정이 가능합니다



품질 향상

일관된 아키텍처와 컨벤션으로
유지보수성과 확장성이 향상됩니다



핵심 포인트

Claude Code는 각자의 PC에서 독립적으로 동작하지만, CLAUDE.md와 design.md를 통해 동일한 컨텍스트와 설계 원칙을 공유하여
일관된 설계 결정과 높은 코드 품질을 유지합니다.

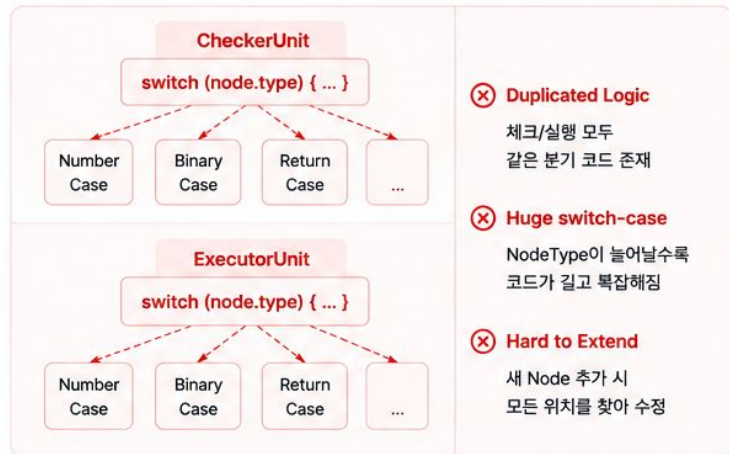
BP#3. Refactoring

1

Refactoring: Visitor Pattern 적용

switch-case 기반 코드 제거 → Visitor Pattern으로 구조 개선

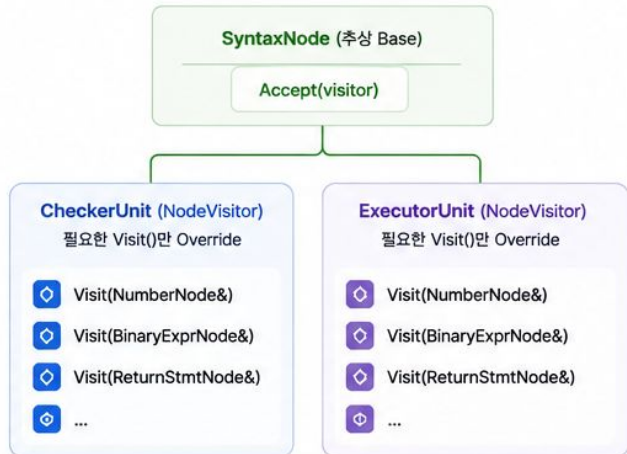
Before



Refactoring

GoF
Visitor Pattern
적용

After



BP#3. Refactoring

2

Impact: 실제 프로젝트에서 검증된 효과

Visitor Pattern 적용 후 다양한 기능 추가 시 기존 코드 수정 없이 빠르게 확장



OCP 준수

Open Closed Principle

새 노드 추가 시
기존 코드 수정 없이
확장 가능



Type Safety

접파일 타입 보장

더블 디스패치 기반으로
정확한 타입의 Visit()가
호출됨



유지보수성 향상

변경 범위 최소화

관심 있는 Visitor만
수정하면 되어
회귀 위험이 낮아짐



개발 효율 향상

빠른 기능 확장

새 기능이 필요한 곳에만
구현하여 개발 속도 향상

실제 기능 개발 사례

Feature #54



Array
...

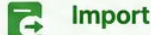
CheckerUnit 
ExecutorUnit 

No Change Needed

수정 불필요



Feature #80



Import
...

CheckerUnit 
ExecutorUnit 

Only Checker Updated

Checker만 수정



Feature #71



Class
...

CheckerUnit 
ExecutorUnit 

Only Visit() Added

필요한 Visit()만 추가



결론

Visitor Pattern 적용 이후, Array / Import / Class 등 다양한 기능 추가 시
기존 switch-case 수정 없이 필요한 Visitor에만 Visit()을 추가하는 방식으로
안전하고 빠르게 확장할 수 있었습니다.

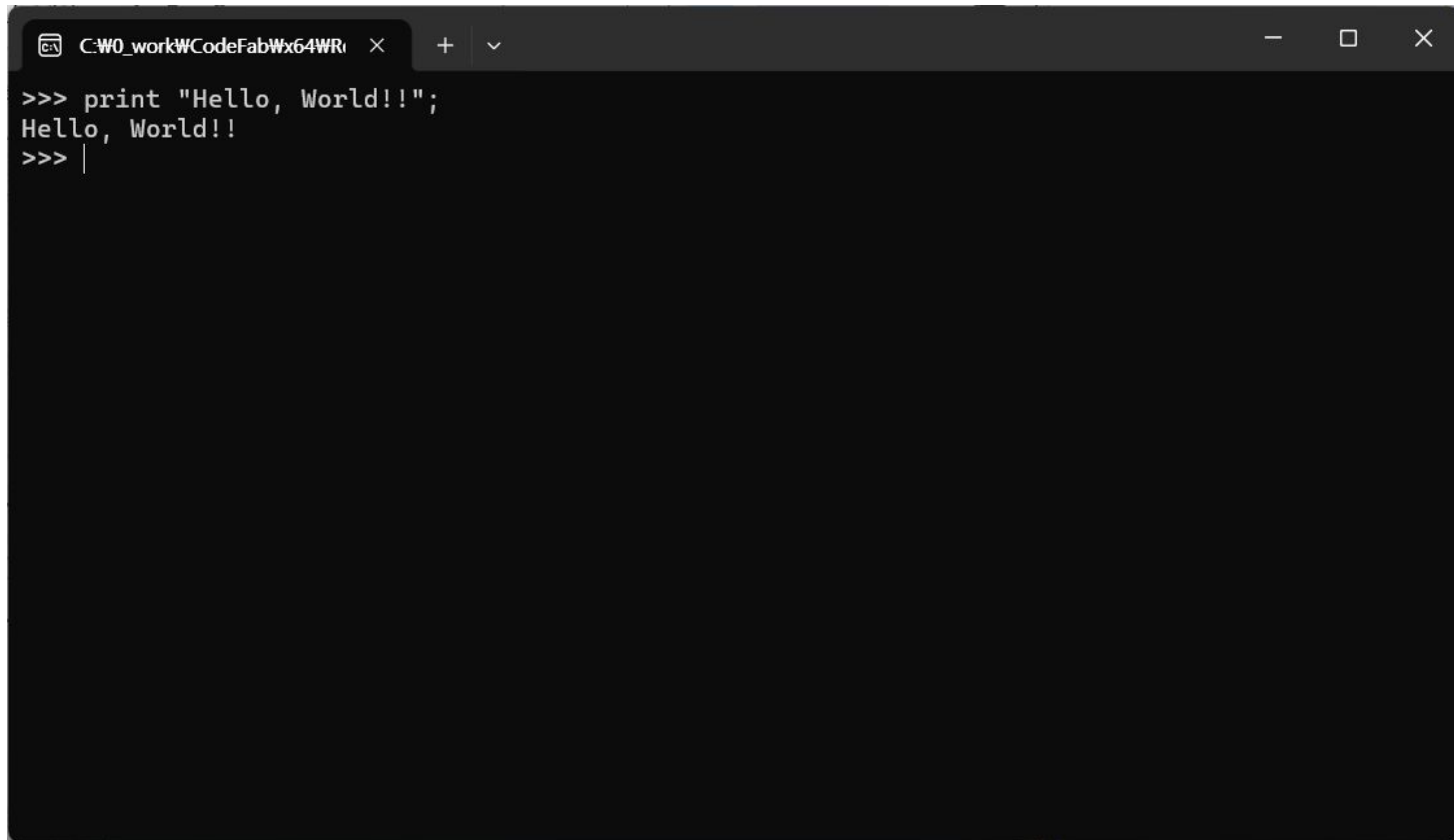


검증된 리팩토링
지속 가능한 설계

BP#3. Refactoring

패턴	적용 위치	설명
Pipeline	전체 구조	Tokenize -> Parse -> Check -> Execute
Registry	StatementParserRegistry	TokenType별 parser factory 등록/조회
Factory	StatementParserRegistrar	parser 객체 생성 책임 캡슐화
Strategy	IStatementParser 구현체	statement 종류별 parsing strategy
Composite	SyntaxNode::children	AST를 트리 구조로 표현
Visitor	NodeVisitor	AST 노드별 동작 분리
Singleton	StatementParserRegistry::Instance()	parser registry 전역 접근

DEMO



A screenshot of a Python REPL window. The window has a dark gray title bar with a single tab labeled 'C:\W0_work\Code\FabWx64\WR...'. The main area is black with white text. The prompt '>>>' is followed by the command 'print "Hello, World!!";'. The output 'Hello, World!!' is displayed on the next line. The prompt '>>>' is followed by a vertical bar '|' on the third line, indicating the cursor is ready for the next command.

```
>>> print "Hello, World!!";  
Hello, World!!  
>>> |
```

Our History...



Code Review in 5 Days

2026-07-06 ~ 2026-07-10 | 집중적인 코드 리뷰로 더 높은 품질을 만들어냈습니다!



92 Commits

5일간 생성된 총 커밋 수



231 Review Comments

5일간 작성된 총 리뷰 코멘트 수



PR Activity

지속적인 PR 생성과 리뷰로
빠르게 피드백을 반영했습니다



Commits / Day



Review Comments / Day



Code Review Workflow



Our Highlights



Fast Feedback

하루 평균 18.4개
코멘트로 빠른 피드백!



Continuous Review

매일 꾸준한 리뷰 활동으로
품질을 향상!



Collaborative Development

함께 리뷰하고 함께 성장!

THE END

5일간의 고된 여정,
수고 많으셨습니다! 동쾌한 성공, 약간의 아쉬움. 



ALL TEST PASSED!!

★ 모든 테스트가 성공적으로 통과되었습니다. ★



도전

5일간의 고된 여정



협업

집중적인 팀워크



개발

꾸준한 개선과 구현



검증

철저한 테스트



완성

CodeFab Interpreter



함께 만들어낸 값진 결과, 감사합니다! 